UPN

UNIVERSIDAD
PRIVADA
DEL NORTE

UPN
UNIVERSIDAD PRIVADA DEL NORTE

UPN, PASIÓN POR TRANSFORMAR VIDAS

Facultad: Ingeniería

Curso: Ingeniería de Software

Carrera profesional: Ingeniería de Software

Ciclo: IV

TEMA 5
Modelado Conceptual con Diagramas de Clases
Ítems 5.3 y 5.4

SEMANA 1 DEL TEMA

DOCENTES:

- Dr. Eric Gustavo Coronel Castillo
- Ing. Andrés Elías Araujo Acuña

UPN.EDU.PE

Tema 5 · Semana 12

Modelo conceptual, diseño y secuencias

Subtemas 5.3 y 5.4: responsabilidades CU–Clases, diagramas de secuencia, paquetes, herramienta CASE, cohesión, acoplamiento y refactorización.

Pregunta eje: ¿cómo pasamos de “clases del dominio” a “clases que colaboran dentro de una arquitectura”?

Ruta de aprendizaje

Semana 12: del modelo conceptual a la colaboración entre clases

1. **Recuperar el modelo conceptual elaborado en la Semana 11**
2. **Relacionar un caso de uso con responsabilidades de clases**
3. **Distinguir modelo conceptual y modelo de diseño por capas**
4. **Construir una secuencia coherente con operaciones y responsabilidades**
5. **Organizar paquetes, revisar cohesión/acoplamiento y registrar refactorización**

Logro de la sesión

Lo que debe demostrar el estudiante al finalizar la clase

Analiza y mejora el modelo conceptual relacionando casos de uso con responsabilidades de clases, verificando la coherencia del diagrama de secuencia y aplicando criterios básicos de organización por paquetes, cohesión, acoplamiento y refactorización.

Explica qué clases participan en un caso de uso y por qué.
Diferencia clases del dominio y clases de diseño técnico.
Verifica que los mensajes de la secuencia correspondan con operaciones justificadas.
Propone al menos una mejora para reducir dependencia o concentración de responsabilidades.

Recuperar saberes previos

El punto de partida es el borrador conceptual de la Semana 11

Cada equipo revisa su matriz Requisito–Clase y responde:

¿Qué requisito originó cada clase?
¿Qué responsabilidad cumple cada clase?
¿Hay alguna clase sin evidencia en requisitos o casos de uso?
¿Qué caso de uso crítico conviene analizar con un diagrama de secuencia?

Regla de trabajo: una clase sin responsabilidad clara o sin trazabilidad debe revisarse antes de usarla en una secuencia.

Conceptual, diseño y secuencia

Tres niveles que no deben confundirse

Modelo conceptual
¿Qué existe en el dominio?

Paciente · Atención · Receta

→

Modelo de diseño
¿Cómo se organiza la solución?

Controller · Service · Repository · Entity

→

Diagrama de secuencia
¿Cómo colaboran las clases en un caso de uso?

Mensajes ordenados en el tiempo

Idea clave: el modelo conceptual explica el problema; el modelo de diseño explica la solución; la secuencia muestra la colaboración.

5.3 Relación diseño–funcional

De un caso de uso a responsabilidades CU–Clases

Caso de uso

Describe un objetivo observable del actor y sus escenarios principales.

→

Responsabilidad

Indica qué clase debe conocer, coordinar o realizar una acción.

→

Operación

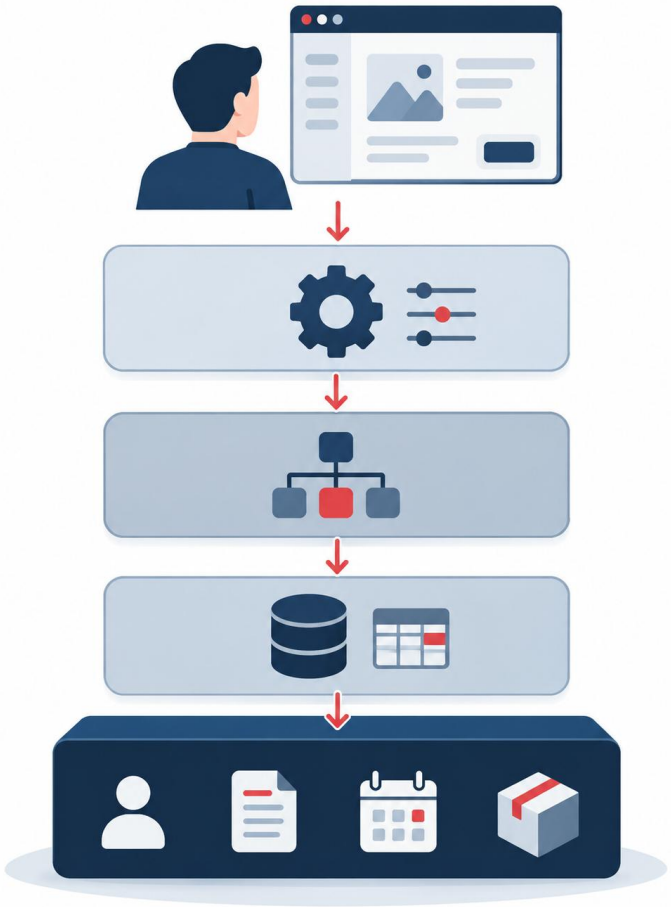
Expresa el servicio que una clase ofrece para cumplir una responsabilidad.

Pregunta guía: ¿el mensaje que aparece en el diagrama de secuencia corresponde a una operación existente o justificadamente incorporada en una clase?

Arquitectura por capas como puente de diseño

Una forma sencilla de organizar responsabilidades

Actor / Interfaz	Usuario o sistema externo que inicia el caso de uso
Controller	Recibe la solicitud y coordina la entrada al caso de uso
Service	Contiene reglas de aplicación y orquesta entidades
Repository	Accede y persiste información
Entity / Dominio	Representa conceptos centrales del problema



Advertencia didáctica: no se reemplaza el dominio por capas. Primero se entiende el problema; luego se organiza la solución.

Ejemplo: Registrar atención

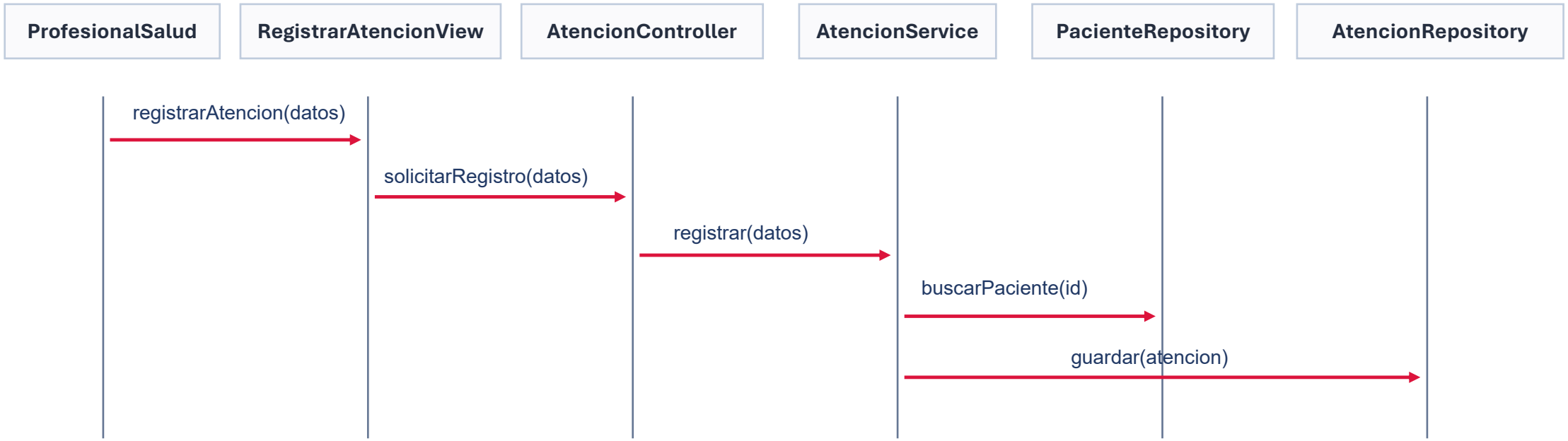
Del caso de uso a clases participantes

Clase / componente	Responsabilidad
AtencionView o RegistrarAtencionView	Captura los datos ingresados por el usuario y muestra resultados o mensajes.
AtencionController	Recibe la solicitud desde la interfaz y deriva la operación al servicio.
AtencionService	Valida reglas y coordina el registro de la atención.
PacienteRepository	Consulta la información del paciente.
AtencionRepository	Persiste la atención registrada.
Paciente / Atencion	Representan conceptos del dominio.

Criterio de coherencia: si AtencionService envía guardarAtencion(), entonces debe existir una operación equivalente en la clase responsable.

Secuencia por capas

La colaboración se lee de izquierda a derecha y de arriba hacia abajo



La secuencia no debe mostrar “todo”, sino la colaboración necesaria para cumplir el caso de uso crítico.

Control de coherencia

Cada mensaje debe tener respaldo en una responsabilidad u operación

Lista de verificación para revisar una secuencia:

¿El actor inicia el caso de uso correcto?
¿Cada objeto participante tiene una responsabilidad justificable?
¿Los mensajes corresponden con operaciones existentes o justificadas?
¿La secuencia contradice el diagrama de clases?
¿Aparecen clases técnicas necesarias para la arquitectura adoptada?
¿Se evita que la entidad de dominio asuma tareas de interfaz o persistencia?

Pregunta crítica: ¿qué error sería más grave: olvidar una clase técnica o asignar una responsabilidad del negocio a la clase equivocada?

Cuestionando la herramienta

Un diagrama de secuencia también puede ocultar decisiones

Si solo modelamos el flujo ideal, invisibilizamos excepciones y usuarios con dificultades.
Si todo depende de un servicio único, aumentan el costo de cambio y la deuda técnica.
Si el diseño prioriza rapidez administrativa, puede olvidar accesibilidad, auditoría o privacidad.
Si la capa técnica domina el análisis, el modelo puede perder contacto con las necesidades reales.

Pregunta para el equipo: ¿qué actor queda peor atendido si el diseño se implementa exactamente como está modelado?

5.4 CASE y estándares de diseño UML

La herramienta ayuda, pero no reemplaza el razonamiento

Herramienta CASE

Facilita diagramar, ordenar elementos, exportar y versionar modelos.

Notación UML

Da un lenguaje común para clases, secuencias, paquetes y relaciones.

Criterio técnico

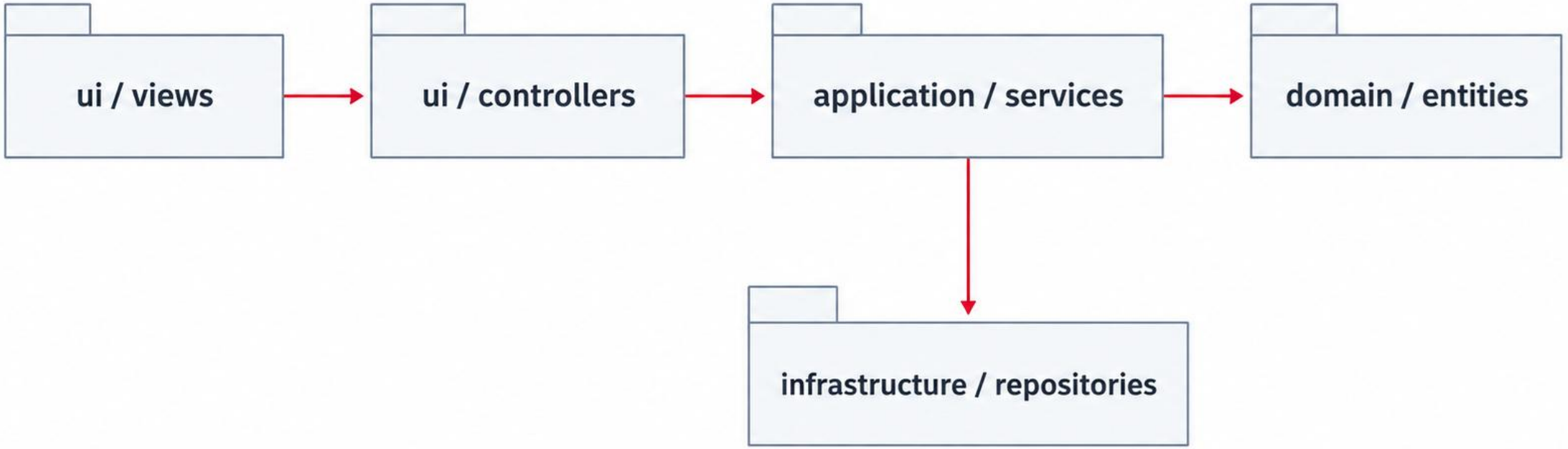
Permite justificar si el modelo es coherente, mantenible y trazable.

Buenas prácticas mínimas:

Usar nombres consistentes con el dominio y con los casos de uso.
Mantener multiplicidades, operaciones y mensajes alineados.
Exportar diagramas legibles y conservar versiones del modelo.
Registrar cambios relevantes con evidencia antes/después.

Diagrama de paquetes

Organizar clases para reducir confusión y dependencia



Las dependencias deben tener sentido y evitar ciclos innecesarios.

Cohesión y acoplamiento

Dos criterios para revisar la calidad estructural

Alta cohesión

Una clase o paquete agrupa responsabilidades relacionadas.

Ejemplo: AtencionService coordina reglas de atención, no genera reportes ni maneja interfaz.

Bajo acoplamiento

Una clase depende de pocas clases y mediante relaciones claras.

Ejemplo: Controller no accede directamente a la base de datos.

Pregunta crítica: ¿qué pasaría si un cambio pequeño en un requisito obliga a modificar muchas clases?

Refactorización con evidencia

Mejorar el modelo sin cambiar el objetivo del caso de uso

Antes

AtencionController valida reglas, busca paciente, guarda atención y actualiza stock.

Problema: baja cohesión y alto acoplamiento.



Después

AtencionController delega.
AtencionService coordina reglas.
Repository persiste.
Entidades conservan lógica del dominio.

Registro mínimo: problema detectado → cambio realizado → criterio técnico → evidencia antes/después.

Actividad vivencial

Revisar y mejorar el modelo del equipo

Seleccione un caso de uso crítico del proyecto.
Construya una matriz CU–Clases: actor, responsabilidad, clase, operación.
Elabore una secuencia inicial con clases de diseño y clases del dominio.
Organice las clases en paquetes y revise dependencias.
Identifique una mejora de cohesión o acoplamiento y documente la refactorización.

Producto de clase: secuencia revisada + paquetes + registro de refactorización antes/después.

Cierre de la Semana 12

Cocrear conclusiones y preparar el Tema 6

Conclusiones esperadas:

La trazabilidad permite evaluar si el diseño responde a requisitos reales.
El modelo conceptual y el modelo de diseño cumplen propósitos distintos.
La secuencia evidencia colaboración, no solo orden de pantallas.
Cohesión, acoplamiento y refactorización ayudan a justificar decisiones de diseño.

Actividad asincrónica: preparar el Tema 6 sobre RUP, fases, roadmap de iteraciones, artefactos por fase y uso de UML dentro de RUP.

Referencias

Fuentes verificables para profundizar

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1

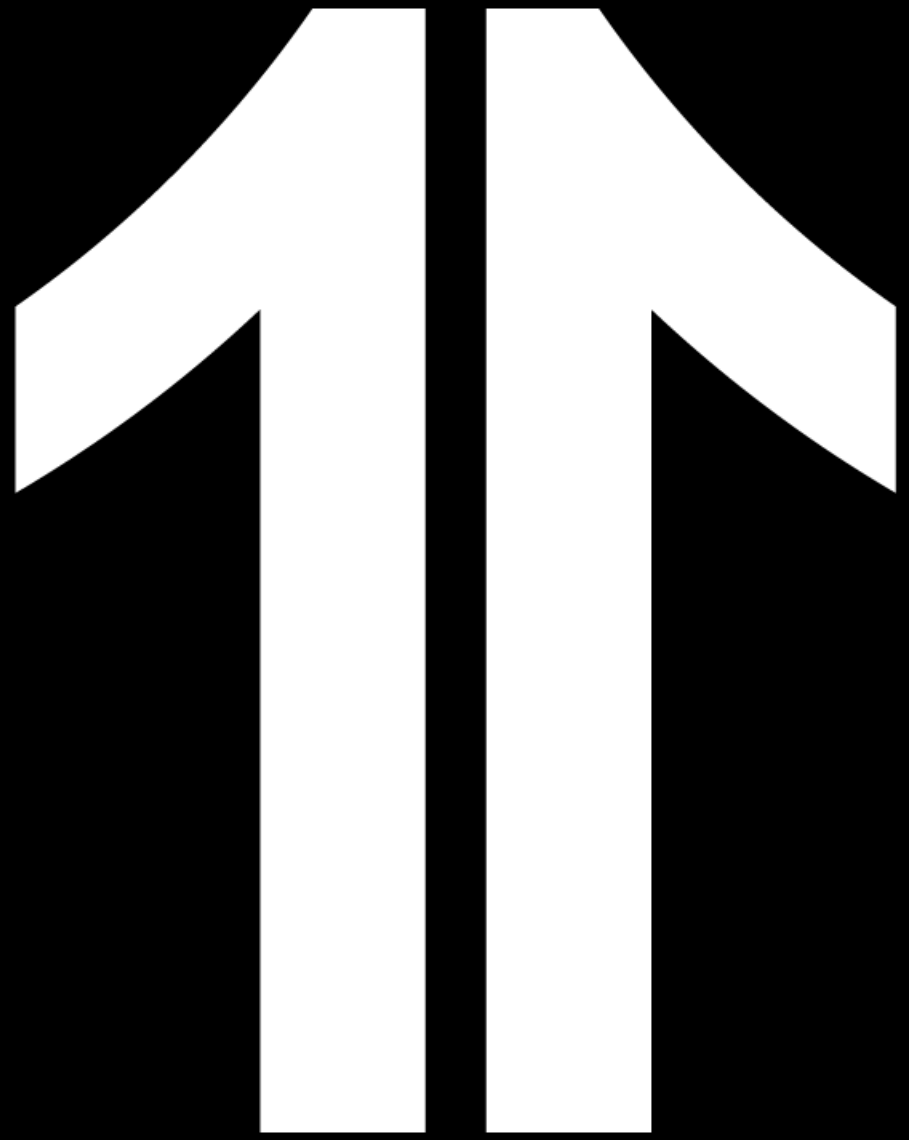
ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering.

Pressman, R. S. (2021). Ingeniería de software: Un enfoque práctico. McGraw-Hill.

Kruchten, P. (2003). The Rational Unified Process: An Introduction (3rd ed.). Addison-Wesley.

Fowler, M. (2003). UML Distilled: A Brief Guide to the Standard Object Modeling Language (3rd ed.). Addison-Wesley.

Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.



GRACIAS


UPN
UNIVERSIDAD
PRIVADA
DEL NORTE